

Semestrálne zadanie

Ekaterina Salova, ID: 121132

Komunikácia s využitím UDP protokolu

Obsah

Uvod	3
Štruktúra hlavičiek protokolu.....	3
• Message Type	3
• Seq Num.....	4
• CRC (Kontrolný súčet).....	4
• Payload Length (Dĺžka dát).....	4
Opis metódy na kontrolu integrity prenesenej správy	4
CRC.....	4
CRC32.....	5
Opis metódy na zabezpečenie spoľahlivého prenosu dát	5
ARQ.....	5
Stop-and-Wait.....	5
Opis metódy na udržanie spojenia	7
Keep-Alive	7
Diagramy opisujúce predpokladané správanie uzla/uzlov	10
• Sekvenčný diagram	11
• Aktivita diagram	12
• Stavový diagram	13
Záver	13

Uvod

Tento projekt implementuje sieť peer-to-peer (P2P) založenú na protokole UDP. Účelom tohto programu je prenášať správy a súbory medzi sieťovými uzlami pomocou základných princípov sieťovej komunikácie, ako je fragmentácia dát, potvrdenie prenosu (ACK), kontrola integrity dát pomocou CRC32 a mechanizmus Keep-Alive na udržanie komunikácie.

Program obsahuje tieto hlavné funkcie:

- **Odosielanie správ:** Používateľ môže posilať textové správy, ktoré sú voliteľne rozdelené na fragmenty, preposielané ďalej a potom zhromaždené v prijímacom uzle.
- **Prenos súborov:** Súbory sa prenášajú po častiach, aby predaj pracoval s veľkým množstvom údajov.
- **Kontrola integrity:** Mechanizmus CRC32 sa používa na overenie údajov na prijímacom konci.
- **Spracovanie chýb a opakovaný prenos:** V prípade straty alebo poškodenia fragmentu sa vykoná opakovaný prenos.
- **Keep-Alive:** Mechanizmus na udržiavanie spojenia nažive a zisťovanie prerušených spojení.
- **Škálovateľnosť:** Schopnosť zmeniť veľkosť fragmentov na optimalizáciu prenosu údajov v závislosti od siete.

Tento projekt je užitočným príkladom použitia soketov v Pythone na vybudovanie spoľahlivej komunikácie medzi uzlami s minimálnou úrovňou abstrakcie, vďaka čomu je flexibilný a prispôsobiteľný pre rôzne úlohy.

Nasledujúce časti podrobne popisujú štruktúru a funkčnosť programu.

Štruktúra hlavičiek protokolu

Protokol používaný v tejto P2P aplikácii používa vlastnú štruktúru hlavičiek na zabezpečenie integrity prenášaných údajov, ich identifikáciu a správne usporiadanie fragmentov. Hlavička je binárny formát, ktorý sa vytvára pomocou štruktúry `struct.pack()` v Python. Formát hlavičky je nasledovný:

Hlavička má celkovú veľkosť 11 bajtov a obsahuje nasledujúce polia:

- **Message Type**

Tento 1-bajtový indikátor určuje, o aký typ správy ide:

- 0x01 (SYN): Dátový fragment.
- 0x02 (ACK): Potvrdenie prijatia fragmentu.
- 0x03 (FIN): Keep-Alive alebo ukončenie spojenia.
- 0x04 (KEEPALIVE): Požiadavka na opakované odoslanie chýbajúceho fragmentu

- Seq Num

Používa sa na identifikáciu fragmentu v rámci celého prenosu. Toto číslo umožňuje správne usporiadanie a detekciu chýbajúcich fragmentov na strane príjemcu.

- CRC (Kontrolný súčet)

Kontrolný súčet CRC32 (4 bajty) je generovaný z dátovej časti balíka (payload). Na strane príjemcu sa vypočíta CRC z prijatých údajov a porovná sa s hodnotou v hlavičke. Ak sa hodnoty nezhodujú, fragment sa považuje za poškodený a odosielateľ je požiadaný o jeho opätovné odoslanie

- Payload Length (Dĺžka dát)

Tento 4-bajtový údaj špecifikuje veľkosť dátovej časti (payload) v bajtoch. Pomáha príjemcovi vedieť, koľko dát očakáva v rámci daného balíka.

Hlavička sa generuje v metóde `build_packet()` nasledovne:

```
header = struct.pack('!BHII', message_type, seq_num, crc, len(payload))
```

Opis metódy na kontrolu integrity prenesenej správy

Raz z najbežnejších metód na zabezpečenie integrity je použitie cyklického redundantného kódu (CRC). Táto metóda generuje kontrolnú hodnotu (tzv. kontrolný súčet) na základe prenášaných údajov. Na prijímacej strane sa táto hodnota znovu vypočíta a porovná s hodnotou uvedenou v hlavičke správy.

CRC

CRC (Cyclic Redundancy Check) je matematická metóda na detekciu chýb v prenášaných dátach. Jej základné princípy sú:

1. Vysielateľ považuje prenášané dáta za veľké binárne číslo.
2. Na toto číslo aplikuje **delenie polynómom** (špeciálny matematický postup), pričom výsledkom je zvyšok po delení – ten sa pridá na koniec dát vo forme **kontrolného súčtu**.
3. Prijímateľ použije rovnaký polynóm na prijaté dáta a kontrolný súčet, aby overil, či zvyšok po delení je nula. Ak nie je nula, dáta boli poškodené.

CRC je obzvlášť výhodný, pretože dokáže efektívne odhaliť jednopólové a viacnásobné chyby v dátach.

CRC32

CRC32 je konkrétny variant CRC s 32-bitovým kontrolným súčtom. Používa sa v mnohých štandardoch, napríklad v ZIP, Ethernet, či PNG. Využíva pevne definovaný polynóm:

Polynóm CRC32: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

1. Generovanie CRC32 kontrolného súčtu **na strane odosielateľa**: Na údaje sa aplikuje algoritmus CRC32, ktorý vygeneruje 32-bitový kontrolný súčet. Tento súčet sa pridá k hlavičke balíka.

```
crc = zlib.crc32(payload)
header = struct.pack('!BHII', message_type, seq_num, crc, len(payload))
```

2. Overenie CRC32 **na strane prijímateľa**: Prijímateľ získa údaje a oddelí ich od kontrolného súčtu. Vypočíta nový CRC32 z prijatých dát a porovná ho s hodnotou v hlavičke. Ak sa hodnoty nezhodujú, údaje sa považujú za poškodené.

```
computed_crc = zlib.crc32(payload)
if computed_crc != crc: # Ak sú dáta poškodené, odošle sa NACK
    print(f"[Poškodené údaje z {addr}], vyžadujúce opätovný prenos]")
```

Opis metódy na zabezpečenie spoľahlivého prenosu dát

ARQ

Automatic Repeat reQuest (ARQ) je technika používaná na zabezpečenie spoľahlivého prenosu dát v situáciách, kde samotný transportný protokol, ako je napríklad UDP (User Datagram Protocol), nezaručuje doručenie správy. ARQ pracuje na princípe opätovného odosielania správ, ktoré neboli úspešne doručené.

Po odoslaní správy odosielateľ čaká na **potvrdenie** od príjemcu (tzv. ACK), že správa bola úspešne prijatá. Ak toto potvrdenie nepríde do vopred stanoveného časového intervalu (tzv. timeout), správa sa považuje za stratenú. V takom prípade sa správa odošle znova. Tento proces sa opakuje, kým sa od príjemcu nedostaví potvrdenie o úspešnom doručení, alebo kým sa nevyčerpá určitý počet pokusov. Ak aj po niekoľkých pokusoch nedôjde k potvrdeniu, systém zaznamená chybu a môže ukončiť prenos alebo vykonať ďalšie kroky na obnovenie.

Stop-and-Wait

Odošle sa jeden údaj. Odosielateľ pred odoslaním ďalšieho fragmentu čaká na potvrdenie (ACK) od príjemcu. Ak potvrdenie nedostane, odosielateľ odošle znova.

Go-Back-N

Viaceré fragmenty sa odosielajú postupne bez čakania na potvrdenie. Ak sa v jednom z fragmentov zistí chyba, všetky nasledujúce fragmenty sa znova prenesú.

Selective Repeat

Rovnako ako v prípade Go-Back-N sa odošle niekoľko fragmentov za sebou. Ale iba tie fragmenty, ktoré boli stratené alebo poškodené, sú znova prenesené.

Mám metódu Stop-and-Wait ARQ s prvkami Selective Repeat ARQ

1. Odosielanie jedného fragmentu s čakaním na ACK:

Keď je každý fragment odoslaný, program čaká na potvrdenie (ACK) pred odoslaním ďalšieho fragmentu. Ak potvrdenie nepríde v určenom čase, fragment sa odošle znova.

```
if not self.ack_received.wait(timeout=5): # Potvrdenie očakávame do 5 sekúnd
    print(f"[Neúspešna prenos fragmentu {seq_num}, opakovaný prenos]")
    # Znova odoslať, ak nie je potvrdenie
    for attempt in range(5):
        self.send_fragment(SYN, seq_num, chunk)
        if self.ack_received.wait(timeout=5): # Opäť čakáme na potvrdenie
            break
```

2. Detekcia straty fragmentov:

Program používa kontrolný súčet (CRC32) na overenie integrity údajov. Ak sú dáta poškodené, odošle sa NACK a fragment sa odošle znova.

```
if computed_crc != crc: # Ak sú dáta poškodené, odošle sa NACK
    print(f"[Poškodené údaje z {addr}, vyžadujúce opätovný prenos]")
    Poskodene = True
    nack_packet = self.build_packet(KEEPALIVE, seq_num, b'')
    self.sock.sendto(nack_packet, addr)
    return
```

3. Opätovný prenos iba jedného fragmentu:

Ak dôjde k strate ACK alebo údajov, prenesie sa iba časť, ktorá nebola potvrdená, nie celý súbor.

4. Správa časového limitu:

Ak počas niekoľkých pokusov nedôjde k potvrdeniu, fragment sa považuje za stratený, čo iniciuje opakované pokusy o prenos.

```
for attempt in range(5):
    self.send_fragment(SYN, seq_num, chunk)
    if self.ack_received.wait(timeout=5): # Opäť čakáme na potvrdenie
        break
    print(f"[Opakovaný prenos fragmentu {seq_num}, pokus {attempt + 1}]")
```

Opis metódy na udržanie spojenia

Keep-Alive

Keep-Alive je mechanizmus používaný na udržanie aktívneho spojenia medzi dvoma počítačmi, aj keď medzi nimi neprebieha aktívna výmena dát. Tento prístup je dôležitý najmä pri dlhodobých pripojeniach cez nespoľahlivé protokoly, ako je UDP, ktoré samé o sebe nezaručujú nepretržité spojenie.

V pravidelných intervaloch, zvyčajne každých pár sekúnd, jeden z počítačov odošle druhému špeciálnu **požiadavku na overenie pripojenia**. Tieto krátke signály neobsahujú užívateľské dáta, ale základné informácie, ako je typ správy, identifikátor pripojenia, a hlavička, ktorá signalizuje, že pripojenie je stále aktívne. Ak prijímajúci počítač odpovie na túto požiadavku, znamená to, že spojenie je stále funkčné.

Ak jeden z počítačov **nedostane odpoveď** v stanovenom časovom limite, interpretujú to ako prerušenie spojenia, čo môže naznačovať, že druhý počítač je offline alebo došlo k technickým problémom. Následne sa systém buď pokúsi spojenie obnoviť, alebo ho ukončí, čím sa zabezpečí, že sa nebudú udržiavať neplatné alebo neaktívne spojenia zbytočne dlho.

1. Pravidelné posielanie správ „Keep-Alive“:

- Program každých **5 sekúnd** posieľa paket typu **FIN** bez užitočného zaťaženia:
`self.send_fragment(FIN, 0, b'')`
- Tieto správy informujú druhý uzol, že spojenie je stále aktívne.

2. Počítanie zmeškaných správ Keep-Alive:

- Program sleduje, či dostal odpoveď (ACK) na svoje Keep-Alive správy. Ak odpoveď nepríde, počíta **zmeškané udržiavacie správy**:
`self.missed_heartbeats += 1`
- Maximálny počet povolených zmeškaných správ je nastavený na **3**:
`self.max_missed_heartbeats = 3`

3. Obnovenie spojenia:

- Ak program nezíska odpoveď počas maximálneho počtu povolených vynechaní, pokúsi sa obnoviť spojenie:
`if self.missed_heartbeats >= self.max_missed_heartbeats:`
`print("[Spojenie sa stratilo. Pokus o obnovenie...])`
- Posieľa ďalšie správy Keep-Alive v intervale a čaká na odpoveď:
`for _ in range(self.max_missed_heartbeats):`
`self.send_fragment(FIN, 0, b'')`

```

time.sleep(self.keep_alive_interval)

if self.missed_heartbeats == 0:

    print("[Spojenie obnovene]")

```

Lua skript vo Wiresharku

Skript vykonáva nasledujúce funkcie: Rozpoznáva užívateľský protokol na základe portu, Dekóduje jednotlivé polia užívateľského protokolu, Zobrazuje údaje protokolu v čitateľnom formáte v okne analýzy Wireshark, Farba zvýrazní správy s užitočným zaťažením (údaje) a servisnými správami.

```

-- Vytvorenie nového protokolu
my_proto = Proto("myp2p", "My P2P Protocol")

-- Definovanie polí protokolu
local f_message_type = ProtoField.uint8("myp2p.type", "Message Type", base.DEC, {
    [1] = "SYN",
    [2] = "ACK",
    [3] = "FIN",
    [4] = "KEEPALIVE"
})
local f_seq_num = ProtoField.uint16("myp2p.seq", "Sequence Number", base.DEC)
local f_crc = ProtoField.uint32("myp2p.crc", "CRC32", base.HEX)
local f_payload_length = ProtoField.uint32("myp2p.length", "Payload Length", base.DEC)
local f_payload = ProtoField.bytes("myp2p.payload", "Payload")

-- Pridanie polí do protokolu
my_proto.fields = {f_message_type, f_seq_num, f_crc, f_payload_length, f_payload}

-- Definovanie funkcií disektorov
function my_proto.dissector(buffer, pinfo, tree)
    -- Kontrola, či je v balíku dostatok dát
    if buffer:len() < 11 then
        return
    end

    -- Nastavenie protokolu
    pinfo.cols.protocol = "MyP2P"

    -- Vytvorte koreňový strom na zobrazenie obsahu balíka
    local subtree = tree:add(my_proto, buffer(), "My P2P Protocol Data")

    -- Analýza polí hlavičky
    local message_type = buffer(0, 1):uint()
    local seq_num = buffer(1, 2):uint()
    local crc = buffer(3, 4):uint()
    local payload_length = buffer(7, 4):uint()

    subtree:add(f_message_type, buffer(0, 1))
    subtree:add(f_seq_num, buffer(1, 2))
    subtree:add(f_crc, buffer(3, 4))
    subtree:add(f_payload_length, buffer(7, 4))

    -- Rozoberáme užitočné zaťaženie, ak existuje
    if payload_length > 0 and buffer:len() > 11 then
        local payload = buffer(11, payload_length)
        subtree:add(f_payload, payload)

        -- Kontrola, či ide o užitočné zaťaženie alebo servisnú správu
        if message_type == 1 then
            subtree:append_text(" [Data Message]")
            pinfo.cols.info:set(string.format("Data Message (Seq: %d, Len: %d)", seq_num, payload_length))
        elseif message_type == 4 then
            subtree:append_text(" [Keep-Alive Message]")
            pinfo.cols.info:set("Keep-Alive Message")
        else
            pinfo.cols.info:set(string.format("Control Message (Type: %d, Seq: %d)", message_type,
seq_num))
        end
    else
        pinfo.cols.info:set(string.format("Control Message (Type: %d, Seq: %d)", message_type, seq_num))
    end
end

-- Zaregistrujte disektor na konkrétnom porte UDP
local udp_port = DissectorTable.get("udp.port")
udp_port:add(5000, my_proto) -- Nahradte 5000 portom svojho protokolu

```

1.Registrácia protokolu

Vytvorí sa nový protokol a zaregistruje sa v programe Wireshark ako vlastný protokol (v tomto prípade Custom UDP Protocol).

Určujú sa porty, na ktorých bude skript spustený.

2. Definovanie polí protokolu

Protokolové polia ako typ správy, poradové číslo, kontrolný súčet (CRC), dĺžka užitočného zaťaženia a samotné užitočné zaťaženie sa zaznamenávajú a rozpoznávajú.

3. Spracovanie správ

V závislosti od typu správy (SYN, ACK, FIN, KEEPALIVE) skript rozlišuje:

Správy s užitočným zaťažením.

Servisné správy (napríklad potvrdenia alebo žiadosti o ukončenie spojenia).

Rozbalí užitočné zaťaženie a zobrazí ho vo výstupe.

4. Indikácia farby

Používa filtre a pravidlá farieb Wireshark na rozlíšenie medzi správami:

Zelená — správy s užitočným zaťažením.

Červená — poškodené správy alebo správy o ukončení spojenia.

Žltá - servisné správy, ako napríklad KEEPALIVE.

Odosielanie správ:

3	5.000666	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
4	6.685958	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
5	18.001021	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
6	11.686351	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
7	15.001272	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
8	16.416051	127.0.0.1	127.0.0.1	MyP2P	54 Data Message (Seq: 3, Len: 11)
9	16.426271	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 3)
10	16.686647	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
11	20.001775	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
12	21.687413	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
13	25.002528	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
14	26.688252	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
15	30.003220	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
16	31.689180	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
17	32.345055	127.0.0.1	127.0.0.1	MyP2P	73 Data Message (Seq: 4, Len: 30)
18	32.346309	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 4)
19	35.003053	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
20	36.689729	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
21	40.004636	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
22	41.690469	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
23	45.004856	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
24	46.690983	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
25	50.005673	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
26	51.691846	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
27	52.825468	127.0.0.1	127.0.0.1	MyP2P	52 Data Message (Seq: 4, Len: 9)
28	52.826725	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 4)
29	55.005906	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
30	56.692123	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
31	60.006196	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
32	61.692904	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
33	62.111910	127.0.0.1	127.0.0.1	MyP2P	57 Data Message (Seq: 0, Len: 14)
34	62.113076	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 0)
35	65.006424	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
36	66.693801	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
37	70.007079	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
38	71.694173	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
39	72.007350	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)

Nahrávanie súboru:

96	143.970606	192.168.0.55	224.0.0.251	MDNS	75 Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "QM" question
97	143.970963	fe80::cfa7:38b2:7f3...	ff02::fb	MDNS	95 Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "QM" question
98	143.971289	fe80::ec:197f:a47a:...	ff02::fb	MDNS	95 Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "QM" question
99	146.571119	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
100	146.710182	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
101	151.571439	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
102	151.710755	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
103	156.572050	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
104	156.711365	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
105	156.858396	127.0.0.1	127.0.0.1	MyP2P	559 Data Message (Seq: 0, Len: 516)
106	156.858758	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 0)
107	156.859234	127.0.0.1	127.0.0.1	MyP2P	559 Data Message (Seq: 1, Len: 516)
108	156.859512	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 1)
109	156.859861	127.0.0.1	127.0.0.1	MyP2P	559 Data Message (Seq: 2, Len: 516)
110	156.860078	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 2)
111	156.860436	127.0.0.1	127.0.0.1	MyP2P	559 Data Message (Seq: 3, Len: 516)
112	156.860644	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 3)
113	156.860971	127.0.0.1	127.0.0.1	MyP2P	559 Data Message (Seq: 4, Len: 516)
114	156.861173	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 4)
115	156.861525	127.0.0.1	127.0.0.1	MyP2P	125 Data Message (Seq: 5, Len: 82)
116	156.861719	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 2, Seq: 5)
117	161.572550	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
118	161.712563	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)

Stratené spojenie:

77	110.013027	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
78	111.697617	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
79	116.698439	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
80	116.698506	127.0.0.1	127.0.0.1	ICMP	71 Destination unreachable (Port unreachable)
81	121.698892	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
82	121.698936	127.0.0.1	127.0.0.1	ICMP	71 Destination unreachable (Port unreachable)
83	121.707352	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
84	121.707388	127.0.0.1	127.0.0.1	ICMP	71 Destination unreachable (Port unreachable)
85	126.707683	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
86	131.569410	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
87	136.570151	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)

Vyskytla sa chyba:

124	66.816348	192.168.0.55	239.255.255.250	SSDP	473 NOTIFY * HTTP/1.1
125	66.816514	192.168.137.1	239.255.255.250	SSDP	474 NOTIFY * HTTP/1.1
126	66.816608	:::1	ff02::c	SSDP	480 NOTIFY * HTTP/1.1
127	66.816731	fe80::ec:197f:a47a:...	ff02::c	SSDP	500 NOTIFY * HTTP/1.1
128	66.816848	fe80::cfa7:38b2:7f3...	ff02::c	SSDP	501 NOTIFY * HTTP/1.1
129	67.232782	127.0.0.1	239.255.255.250	SSDP	479 NOTIFY * HTTP/1.1
130	67.232952	192.168.0.55	239.255.255.250	SSDP	482 NOTIFY * HTTP/1.1
131	67.233154	192.168.137.1	239.255.255.250	SSDP	483 NOTIFY * HTTP/1.1
132	67.233261	:::1	ff02::c	SSDP	489 NOTIFY * HTTP/1.1
133	67.233410	fe80::ec:197f:a47a:...	ff02::c	SSDP	509 NOTIFY * HTTP/1.1
134	67.233558	fe80::cfa7:38b2:7f3...	ff02::c	SSDP	510 NOTIFY * HTTP/1.1
135	68.563430	127.0.0.1	239.255.255.250	SSDP	526 NOTIFY * HTTP/1.1
136	68.563619	192.168.0.55	239.255.255.250	SSDP	529 NOTIFY * HTTP/1.1
137	68.563806	192.168.137.1	239.255.255.250	SSDP	530 NOTIFY * HTTP/1.1
138	68.563900	:::1	ff02::c	SSDP	536 NOTIFY * HTTP/1.1
139	68.564035	fe80::ec:197f:a47a:...	ff02::c	SSDP	556 NOTIFY * HTTP/1.1
140	68.564144	fe80::cfa7:38b2:7f3...	ff02::c	SSDP	557 NOTIFY * HTTP/1.1
141	68.950596	127.0.0.1	127.0.0.1	MyP2P	43 Control Message (Type: 3, Seq: 0)
142	69.468073	127.0.0.1	127.0.0.1	MyP2P	559 Data Message (Seq: 0, Len: 516)

Vylepšenia

Oproti predchádzajúcej verzii som v novej implementácii výrazne vylepšila funkcionálnosť a pridala nové funkcie. Tu sú hlavné zmeny:

Fragmentácia a prenos dát: V novej verzii som pridal podporu pre fragmentáciu správ, ktorá umožňuje prenášať veľké súbory rozdelením na menšie časti. Každý fragment sa teraz kontroluje pomocou CRC, aby sa zabezpečilo, že údaje nie sú počas prenosu poškodené. Ak sa niektorý fragment stratí alebo poškodí, odošle sa znova pomocou mechanizmu NACK.

Spracovanie a obnova chýb: Na rozdiel od predchádzajúcej verzie, kde sa chyby neošetrovali, nová implementácia pridáva kontrolu integrity údajov pomocou CRC32 a možnosť opätovného odoslania poškodených fragmentov.

Správa pripojenia (keep-alive): V novej verzii som implementoval mechanizmus keep-alive, ktorý pravidelne kontroluje stav pripojenia. Každých niekoľko sekúnd sa odošle správa, ktorá skontroluje, či je pripojenie aktívne. Ak niekoľko udržiavacích správ nedostane odpoveď, uzol sa pokúsi znovu nadviazať

spojenie. Predchádzajúca verzia tento mechanizmus nemala a spojenie jednoducho „viselo“ bez upozornenia na stratu.

Interaktívne menu a nastavenia: V novej verzii som pridal interaktívne menu, ktoré umožňuje užívateľovi konfigurovať parametre prenosu ako veľkosť fragmentu a simuláciu chýb. Teraz si tiež môžete vybrať typ správy (SYN, ACK, FIN atď.), vďaka čomu je interakcia s programom flexibilnejšia. Stará verzia mala iba základnú kontrolu nad odosielaním správ.

Spracovanie typov správ: V novej verzii som rozšíril podporu typov správ: teraz je možné okrem bežných dát odosielať a prijímať správy ako SYN (na spustenie spojenia), ACK (na potvrdenie prijatia), FIN (na dokončenie prenosu) a KEEPALIVE (na kontrolu stavových pripojení). Predchádzajúca verzia podporovala iba základné správy.

Spracovanie vlákien: V novej implementácii som pridal multitasking pomocou vlákien, aby sa úlohy spúšťali asynchrónne. Napríklad proces odosielania údajov a proces prijímania správ sa teraz vykonávajú paralelne, čo urýchľuje spracovanie a zefektívňuje program. V starej verzii bol multitasking obmedzený na základné funkcie.

Jednoduchosť a funkčnosť: Stará verzia mala oveľa jednoduchší kód a bola určená len na základné odovzdávanie správ. Nová verzia výrazne rozšírila možnosti: teraz môžete nielen odosielať správy, ale aj spravovať ich fragmentáciu, kontrolovať ich integritu a tiež udržiavať stabilné spojenie s ostatnými uzlami prostredníctvom udržiavania nažive.

Diagramy opisujúce predpokladané správanie uzla/uzlov

- Sekvenčný diagram

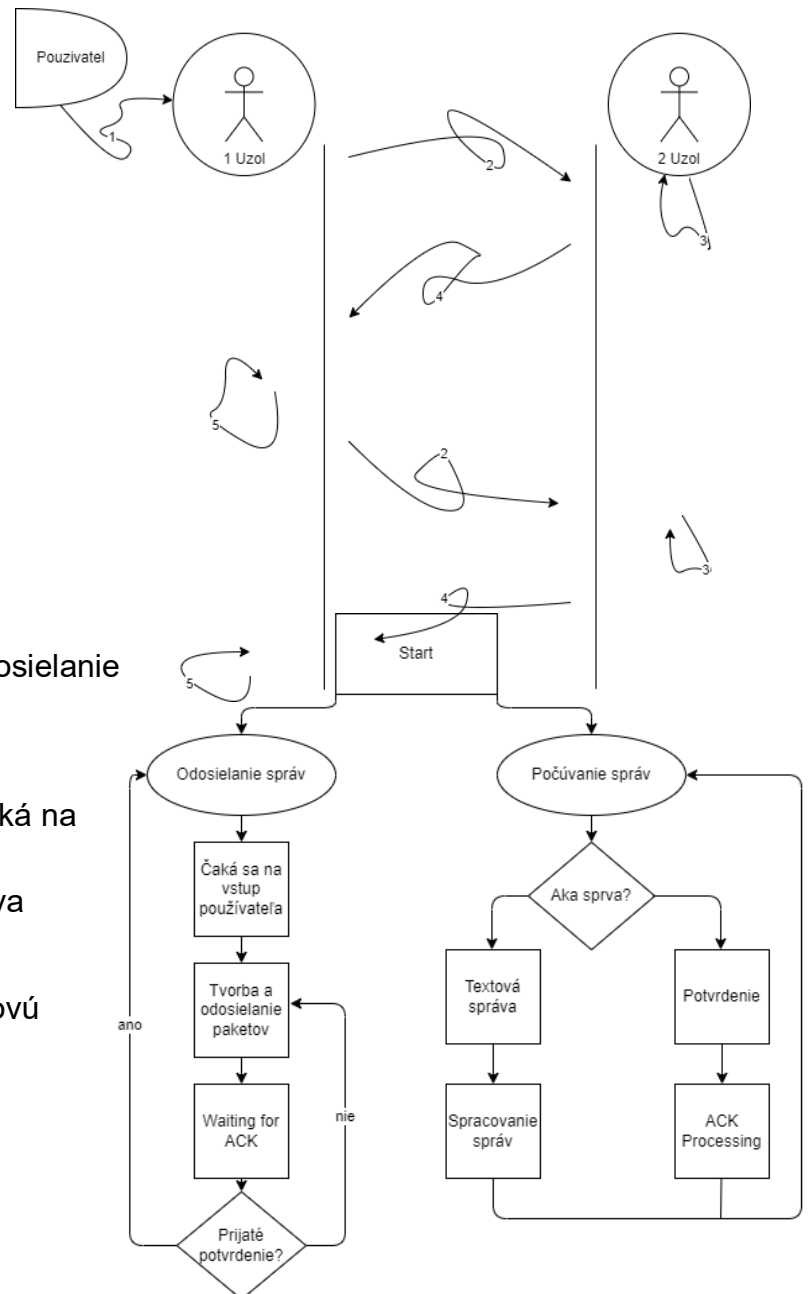
- 1 – Zadávanie správy
 - 2 – Odoslanie balíka
 - 3 – Spracovanie a výstup správy
 - 4 – Odoslať potvrdenie (ACK)
 - 5 – Spracovanie potvrdenia
- A všetko odznova

• Activity diagram

Súčasne bežia 2 pobočky – Odosielanie správ a Počúvanie správ.

Prvá vetva čaká na správu od používateľa, potom ju spracuje a čaká na potvrdenie dokončenia. Ak to prišlo, vrátime sa späť, ak nie, správu znova odošleme.

Počúvanie správ - ak ide o textovú správu, dostaneme ju, ak existuje potvrdenie o prijatí, spracujeme ju a vrátime späť.



- Stavový diagram

Idle (čakanie): Uzol je pripravený na odosielanie alebo prijímanie správ, čaká na aktívnu akciu (vstup od používateľa alebo správu z iného uzla)

Sending: Uzol odošle správu cieľovému uzlu

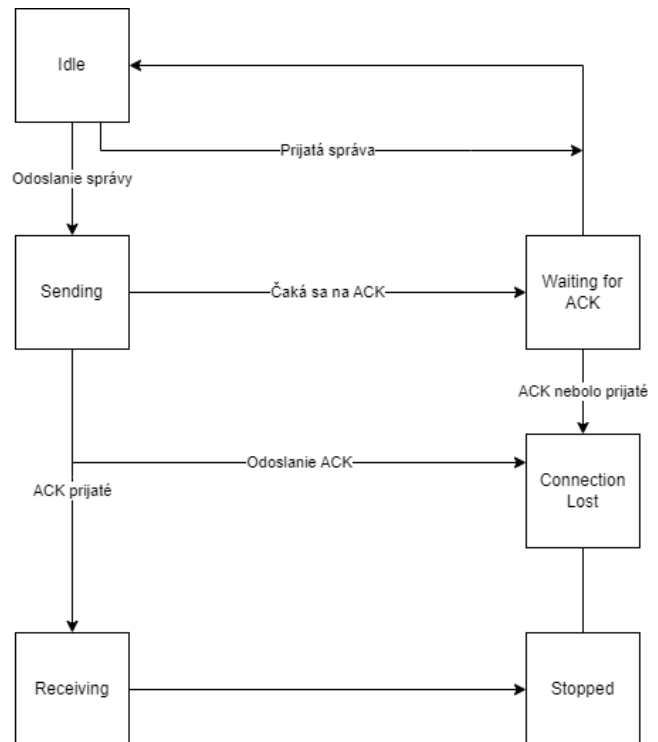
Waiting for ACK: Uzol odoslal správu a čaká na potvrdenie (ACK)

Receiving: Uzol prijme správu z iného uzla

ACK Received: Uzol dostal potvrdenie (ACK)

Connection Lost: Spojenie sa stratí (napríklad ak nebolo prijaté žiadne ACK)

Stopped: Uzol dokončil svoju prácu



Záver

V tejto dokumentácii sme sa podrobne pozreli na návrh a implementáciu P2P aplikácie, ktorá využíva natívny protokol UDP na efektívnu komunikáciu medzi dvoma hostiteľmi v lokálnej sieti. Implementované mechanizmy na zabezpečenie integrity prenosu, opätovné odosielanie stratených správ a metódy údržby pripojenia prispievajú k spoľahlivosti a odolnosti našej aplikácie voči chybám.

Úspešná implementácia a testovanie funkčnosti preukázalo, že náš protokol dokáže zabezpečiť efektívny prenos textu a súborov a zároveň umožňuje simuláciu chýb na testovanie jeho spoľahlivosti.